

第8章 集成学习



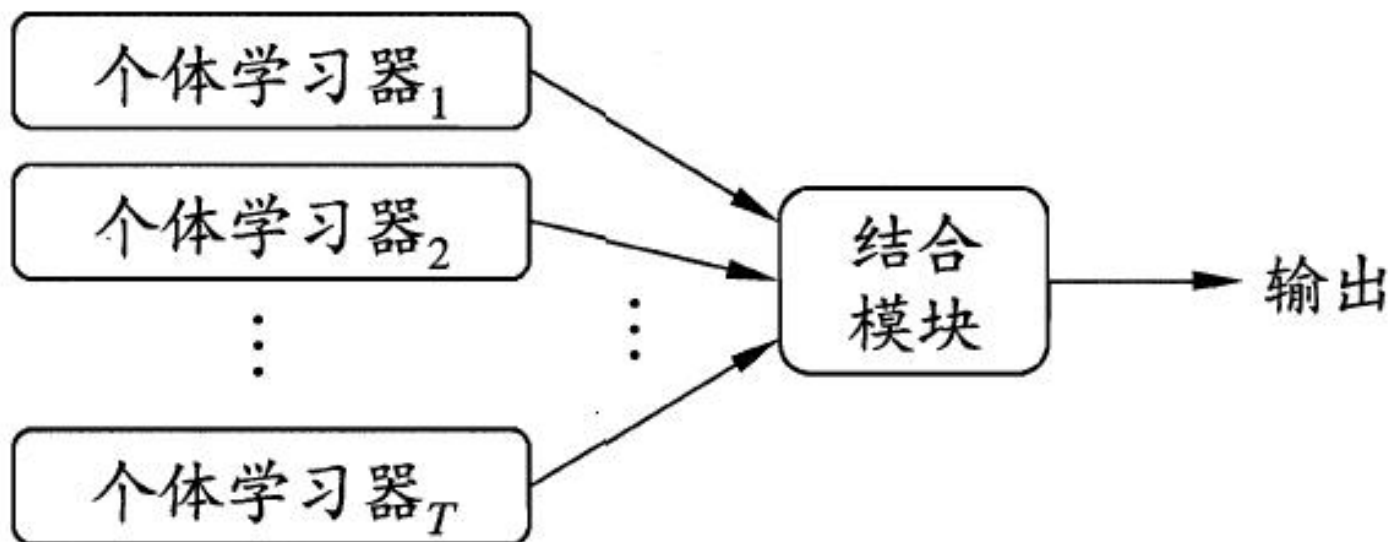
主要内容

- 个体与集成
 - Boosting
 - Bagging与随机森林
 - 结合策略
 - 多样性

8.1 个体与集成

集成学习

- 通过构建并结合多个学习器来完成学习任务
 - 结构：先产生一组个体学习器，再用某种策略将它们结合起来
 - 个体学习器通常由一个现有学习算法从训练数据产生
- 也称为多分类器系统、基于委员会的学习



8.1 个体与集成

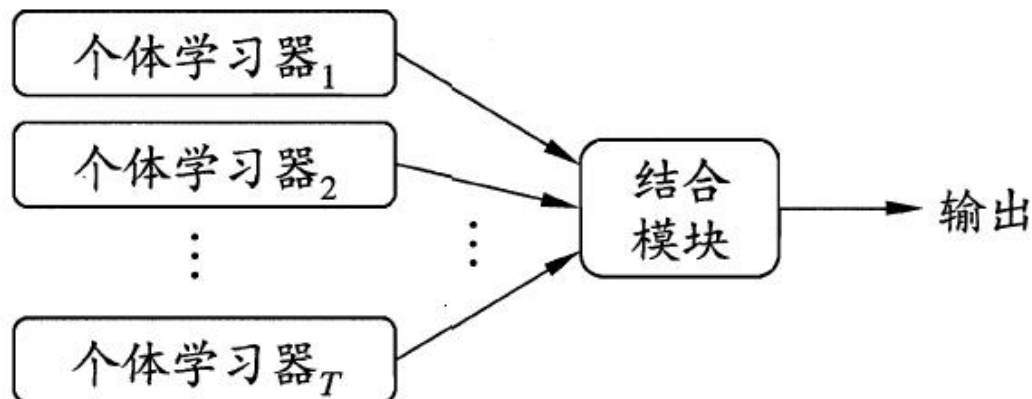
■ 同质与异质集成

■ 同质集成

- 集成中只包含同种类型的个体学习器
- 相应的学习算法称为基学习算法
- 个体学习器亦称基学习器

■ 异质集成

- 个体学习器由不同的学习算法生成
- 不存在基学习算法
- 个体学习器称为组件学习器或个体学习器，不称为基学习器

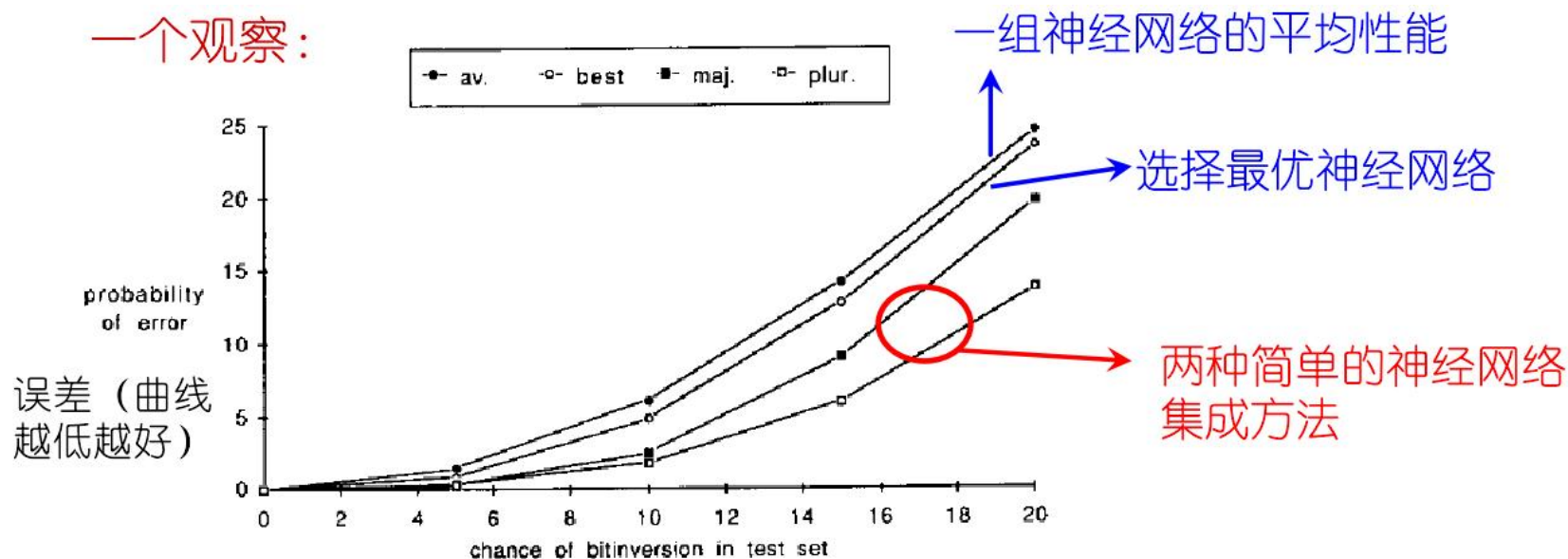


8.1 个体与集成

为什么集成

- 集成的泛化性能通常显著优于单个学习器的泛化性能
 - 对于弱学习器尤为明显
 - 集成学习的很多理论研究都是针对弱学习器进行的，基学习器也被称为弱学习器
 - 实践中希望使用较少的个体学习器，或重用关于学习器的一些经验，常会使用比较强的学习器

一个观察：



8.1 个体与集成

如何集成

例子

- 二分类任务，三个分类器，三个测试样本

- $\checkmark$ 表示分类正确， $\times$ 表示分类错误
- 集成学习结果通过投票法产生，即少数服从多数
- 集成结果

- 集成a，每个分类器只有66.6%的精度，集成学习达到了100%
- 集成b，三个分类器没有差别，集成之后性能没有提高
- 集成c，每个分类器的精度都只有33.3%，集成学习的结果变得更糟

	测试例1	测试例2	测试例3		测试例1	测试例2	测试例3		测试例1	测试例2	测试例3
h_1	$\checkmark$	$\checkmark$	$\times$	h_1	$\checkmark$	$\checkmark$	$\times$	h_1	$\checkmark$	$\times$	$\times$
h_2	$\times$	$\checkmark$	$\checkmark$	h_2	$\checkmark$	$\checkmark$	$\times$	h_2	$\times$	$\checkmark$	$\times$
h_3	$\checkmark$	$\times$	$\checkmark$	h_3	$\checkmark$	$\checkmark$	$\times$	h_3	$\times$	$\times$	$\checkmark$
集成	$\checkmark$	$\checkmark$	$\checkmark$	集成	$\checkmark$	$\checkmark$	$\times$	集成	$\times$	$\times$	$\times$

(a) 集成提升性能

(b) 集成不起作用

(c) 集成起负作用

8.1 个体与集成

■ 如何集成

■ 结论

- 个体学习器应好而不同，即个体学习器要有一定的准确性和多样性，学习器不能太坏且具有差异

■ 分析

- 设二分类问题 $y \in \{-1, +1\}$ 和真实函数 f , T 个基分类器

- 设定基分类器的错误率为 ε ，即对基分类器 h_i 有

$$P(h_i(\mathbf{x}) \neq f(\mathbf{x})) = \varepsilon$$

- 设集成通过简单投票法，则集成分类器为

$$H(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^T h_i(\mathbf{x})\right)$$

- 假设基分类器的错误率相互独立，则由Hoeffding不等式可知集成的错误率为

$$P(H(\mathbf{x}) \neq f(\mathbf{x})) = \sum_{k=0}^{\lfloor T/2 \rfloor} \binom{T}{k} (1-\varepsilon)^k \varepsilon^{T-k} \leq \exp\left[-\frac{1}{2}T(1-2\varepsilon)^2\right]$$

- T 增大，集成的错误率将指数级下降，最终趋于零

8.1 个体与集成

■ 集成学习方法

■ 关键问题

- 现实中个体学习器是为解决同一问题训练出来的，不可能相互独立
- 个体学习器的准确性和多样性存在冲突
 - 准确性很高后，增加多样性就要牺牲准确性
- 产生并结合个体学习器是集成学习研究的核心

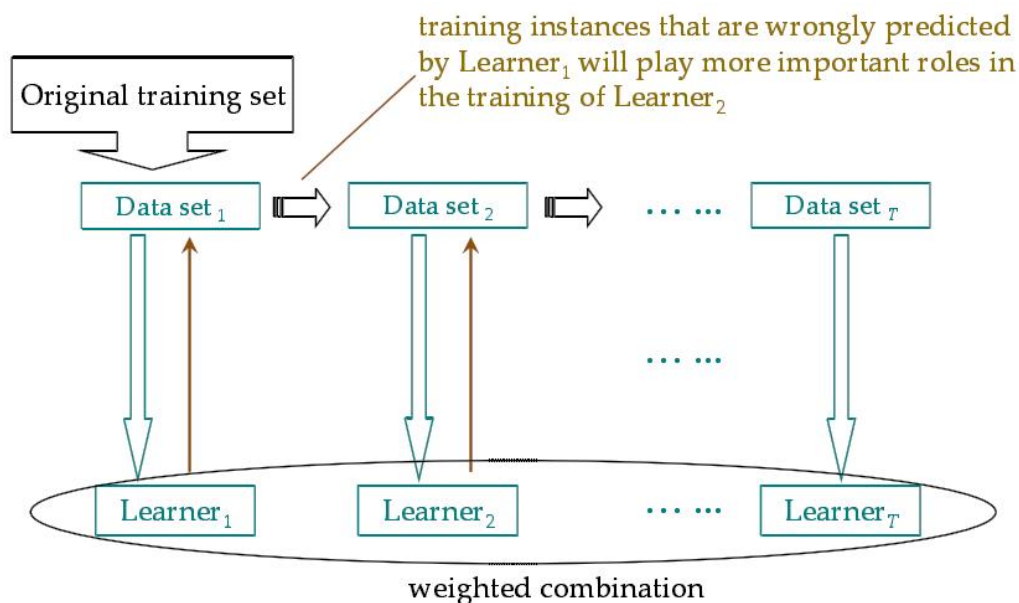
■ 方法分为两大类

- 根据个体学习器的生成方式
 - 序列化方法
 - 个体学习器间存在强依赖关系、必须串行生成
 - 如AdaBoost、Gradient Boost、LPBoost、...
 - 并行化方法
 - 个体学习器间不存在强依赖关系、可同时生成
 - 如Bagging、Random Forest、Random Subspace

8.2 Boosting

工作机制

- 先从初始训练集训练出一个基学习器
- 根据基学习器的表现对训练样本分布调整
 - 使得先前做错的训练样本在后续受到更多关注
- 基于调整后的样本分布训练下一个基学习器
- 如此重复直至基学习器数目达到指定的T值
- 最终将T个基学习器进行加权结合



8.2 Boosting

■ AdaBoost算法

- Boosting族最著名代表，有多种推导方式
- 基于加性模型(基学习器的线性组合)

- 集成学习器模型

$$H(\mathbf{x}) = \sum_{t=1}^T \alpha_t h_t(\mathbf{x})$$

- 最小化指数损失函数

$$l_{\text{exp}}(H|\mathcal{D}) = E_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H(\mathbf{x})}]$$

- 对 $H(\mathbf{x})$ 的偏导

$$\frac{\partial l_{\text{exp}}(H|\mathcal{D})}{\partial H(\mathbf{x})} = -e^{-H(\mathbf{x})} P(f(\mathbf{x})=1|\mathbf{x}) + e^{H(\mathbf{x})} P(f(\mathbf{x})=-1|\mathbf{x})$$

- 令上式为0，得

$$H(\mathbf{x}) = \frac{1}{2} \ln \frac{P(f(\mathbf{x})=1|\mathbf{x})}{P(f(\mathbf{x})=-1|\mathbf{x})}$$

$$\text{sign}(H(\mathbf{x})) = \begin{cases} 1, & P(f(\mathbf{x})=1|\mathbf{x}) > P(f(\mathbf{x})=-1|\mathbf{x}) \\ -1, & P(f(\mathbf{x})=1|\mathbf{x}) < P(f(\mathbf{x})=-1|\mathbf{x}) \end{cases} = \underset{y \in \{-1, 1\}}{\text{argmax}} P(f(\mathbf{x})=y|\mathbf{x})$$

- 达到了贝叶斯最优错误率，说明指数损失函数是分类任务0/1损失函数的替代损失函数

8.2 Boosting

■ AdaBoost算法

■ 加性模型(基学习器的线性组合)

■ 求权重

- 通过初始数据分布得第一个基分类器 h_1 , 此后迭代生成 h_t, α_t
- 基分类器 h_t 基于分布 $\mathcal{D}_t$ 产生后, 权重 α_t 应使得 $\alpha_t h_t$ 最小化指数损失函数

$$\begin{aligned} l_{\text{exp}}(\alpha_t h_t | \mathcal{D}_t) &= E_{\mathbf{x} \sim \mathcal{D}_t} [e^{-f(\mathbf{x}) \alpha_t h_t(\mathbf{x})}] \\ &= e^{-\alpha_t} P_{\mathbf{x} \sim \mathcal{D}_t} (f(\mathbf{x}) = h_t(\mathbf{x})) + e^{\alpha_t} P_{\mathbf{x} \sim \mathcal{D}_t} (f(\mathbf{x}) \neq h_t(\mathbf{x})) \\ &= e^{-\alpha_t} (1 - \varepsilon_t) + e^{\alpha_t} \varepsilon_t \end{aligned}$$

- 令上式 α_t 偏导为0得 $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$

8.2 Boosting

■ AdaBoost算法

■ 加性模型(基学习器的线性组合)

■ 求分布 $\mathcal{D}_t$

■ 获得 H_{t-1} 后样本分布将进行调整

■ 使下一轮的基学习器能纠正一些错误, 即最小化

$$\begin{aligned} l_{\text{exp}}(H_{t-1} + h_t | \mathcal{D}) &= E_{\mathbf{x} \sim \mathcal{D}} [e^{-f(\mathbf{x})(H_{t-1}(\mathbf{x}) + h_t(\mathbf{x}))}] \\ &= E_{\mathbf{x} \sim \mathcal{D}} \left[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} (1 - f(\mathbf{x})h_t(\mathbf{x}) + 1/2) \right] \end{aligned} \quad \text{用泰勒展式}$$

■ 可得理想基学习器

$$\begin{aligned} h_t(\mathbf{x}) &= \underset{h}{\operatorname{argmin}} l_{\text{exp}}(H_{t-1} + h_t | \mathcal{D}) = \underset{h}{\operatorname{argmax}} E_{\mathbf{x} \sim \mathcal{D}} \left[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} f(\mathbf{x})h_t(\mathbf{x}) \right] \\ &= \underset{h}{\operatorname{argmax}} E_{\mathbf{x} \sim \mathcal{D}} \left[\frac{e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}}{E_{\mathbf{x} \sim \mathcal{D}}(e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})})} f(\mathbf{x})h_t(\mathbf{x}) \right] \end{aligned}$$

■ 令 $\mathcal{D}_t(\mathbf{x}) = \frac{\mathcal{D}(\mathbf{x})e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}}{E_{\mathbf{x} \sim \mathcal{D}}(e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})})}$, 有

$$h_t(\mathbf{x}) = \underset{h}{\operatorname{argmax}} E_{\mathbf{x} \sim \mathcal{D}_t} [f(\mathbf{x})h_t(\mathbf{x})] = \underset{h}{\operatorname{argmin}} E_{\mathbf{x} \sim \mathcal{D}_t} [\hat{u}(f(\mathbf{x}) \neq h_t(\mathbf{x}))]$$

8.2 Boosting

AdaBoost算法

- 加性模型(基学习器的线性组合)

- 求分布 $\mathcal{D}_t$

- 可见理想的 h_t 将在分布 $\mathcal{D}_t$ 下最小化分类误差

$$h_t(\mathbf{x}) = \underset{h}{\operatorname{argmax}} E_{\mathbf{x} \sim \mathcal{D}_t} [f(\mathbf{x}) h_t(\mathbf{x})] = \underset{h}{\operatorname{argmin}} E_{\mathbf{x} \sim \mathcal{D}_t} [\mathbb{I}(f(\mathbf{x}) \neq h_t(\mathbf{x}))]$$

- 这在一定程度上类似残差逼近的思想

- 考虑 $\mathcal{D}_t$ 和 $\mathcal{D}_{t+1}$

$$\mathcal{D}_{t+1}(\mathbf{x}) = \frac{\mathcal{D}(\mathbf{x}) e^{-f(\mathbf{x}) H_t(\mathbf{x})}}{E_{\mathbf{x} \sim \mathcal{D}}(e^{-f(\mathbf{x}) H_t(\mathbf{x})})} = \mathcal{D}_t(\mathbf{x}) e^{-f(\mathbf{x}) \alpha_t h_t(\mathbf{x})} \frac{E_{\mathbf{x} \sim \mathcal{D}}(e^{-f(\mathbf{x}) H_{t-1}(\mathbf{x})})}{E_{\mathbf{x} \sim \mathcal{D}}(e^{-f(\mathbf{x}) H_t(\mathbf{x})})}$$

- 实施

- 重赋权法：根据样本分布为每个训练样本重新赋予权重
 - 重采样法：对训练集重新进行采样，用重采样样本集对基学习器进行训练
 - 可获得重新启动机会，避免训练过程早停，持续到预设的T轮
 - 两种做法没有显著的优劣差别

8.2 Boosting

AdaBoost算法

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程:

- 1: $\mathcal{D}_1(\mathbf{x}) = 1/m$. 初始化样本权值分布
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $h_t = \mathcal{L}(D, \mathcal{D}_t)$; 训练分类器
 - 4: $\epsilon_t = P_{\mathbf{x} \sim \mathcal{D}_t}(h_t(\mathbf{x}) \neq f(\mathbf{x}))$; 估计误差
 - 5: **if** $\epsilon_t > 0.5$ **then break**
 - 6: $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$; 确定分类器权重
 - 7:
$$\begin{aligned} \mathcal{D}_{t+1}(\mathbf{x}) &= \frac{\mathcal{D}_t(\mathbf{x})}{Z_t} \times \begin{cases} \exp(-\alpha_t), & \text{if } h_t(\mathbf{x}) = f(\mathbf{x}) \\ \exp(\alpha_t), & \text{if } h_t(\mathbf{x}) \neq f(\mathbf{x}) \end{cases} \\ &= \frac{\mathcal{D}_t(\mathbf{x}) \exp(-\alpha_t f(\mathbf{x}) h_t(\mathbf{x}))}{Z_t} \end{aligned}$$

更新样本分布,
 Z_t 是规范化因子
 - 8: **end for**
- 输出: $H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$

8.2 Boosting

AdaBoost算法

- 从偏差方差分解的角度看，Boosting主要关注降低偏差，因此Boosting能基于泛化性能相当弱的学习器构建出很强的集成
- 例：以决策树桩为基学习器，不同规模集成

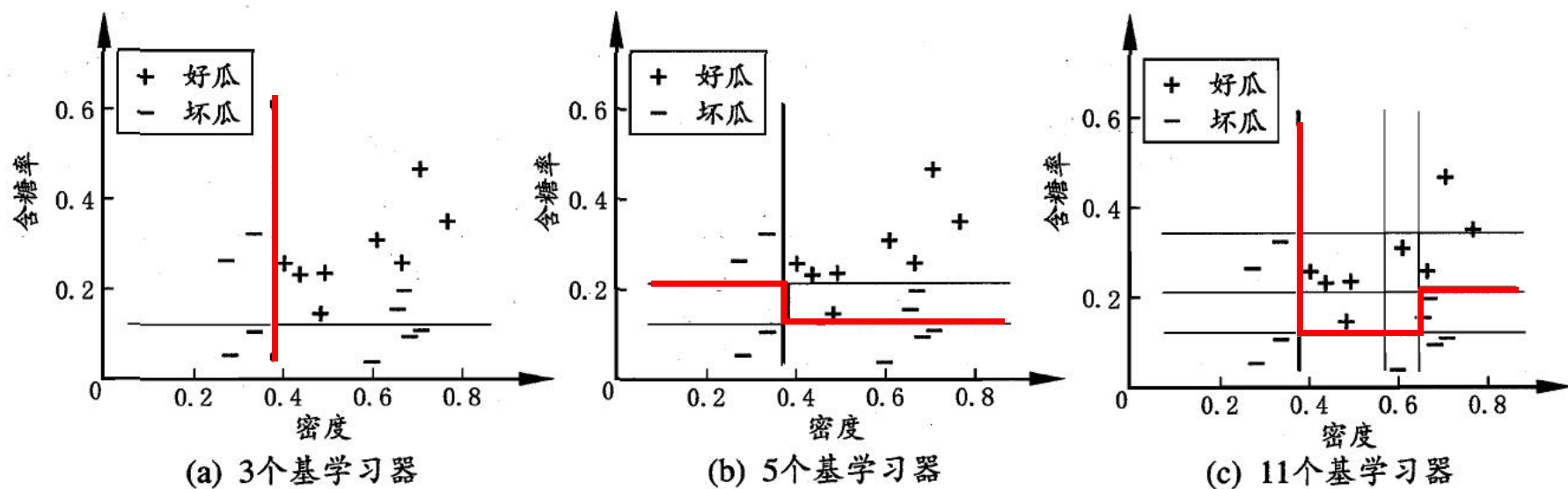


图 8.4 西瓜数据集 3.0α 上 AdaBoost 集成规模为 3、5、11 时，集成(红色)与基学习器(黑色)的分类边界.

8.3 Bagging与随机森林

思想

- 欲得到泛化性能强的集成，个体学习器应尽可能相互独立
- 独立在现实中无法做到，可以设法使基学习器尽可能具有较大的差异
 - 一种可能的方法是对训练样本进行采样，产生若干不同子集，再从每个数据子集训练出基学习器
 - 因训练数据不同，获得的基学习器可能具有较大差异
- 若采样子集完全不同，则每个基学习器只用到一小部分训练数据，可能不足以有效学习
 - 可使用相互有交叠的采样子集

8.3 Bagging与随机森林

8.3.1 Bagging

- 并行式集成学习方法最著名的代表

- 方法

- 基于自助采样法

- 给定包含 m 个样本的数据集，先随机取出一个样本放入采样集中，再将其放回初始数据集
- 经过 m 次随机采样，得到 m 个样本的采样集，初始训练集中有样本在采样集中多次出现，有的则从未出现
- 初始训练集中约有63.2%的样本出现在采样集中

- 采样出 T 个训练样本集，训练出 T 个基学习器，将基学习器进行结合

- 在对预测输出进行结合时，常用简单投票法。对回归任务使用简单平均法

- 若出现两个类收到同样票数的情形
 - 最简单的做法是随机选择一个
 - 考察学习器投票的置信度来确定最终胜者

8.3 Bagging与随机森林

8.3.1 Bagging

■ 算法流程

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程:

1: **for** $t = 1, 2, \dots, T$ **do**

2: $h_t = \mathcal{L}(D, \mathcal{D}_{bs})$ 自助采样产生的样本分布

3: **end for**

输出: $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = y)$

8.3 Bagging与随机森林

8.3.1 Bagging

■ 优势

- 与训练一个学习器的复杂度同阶
 - 假定基学习器的计算复杂度为 $O(m)$ ，则Bagging的复杂度大致为 $(O(m)+O(s))$ ，考虑到采样与投票/平均过程的复杂度 $O(s)$ 很小
 - 是一个很高效的集成学习算法
- 初始训练集会剩下约36.8%样本
 - 可作验证集对泛化性能进行包外估计
 - 基学习器是决策树时，可用来辅助剪枝，或用于估计结点后验概率以辅助对零训练样本结点的处理
 - 基学习器是神经网络时，可用来辅助早期停止减小过拟合风险
- 从偏差方差看，主要降低方差，对不剪枝决策树、神经网络等易受样本扰动的学习器效果明显

8.3 Bagging与随机森林

8.3.1 Bagging

■ 例

- 基于信息增益划分的决策树为基学习器，不同规模的集成

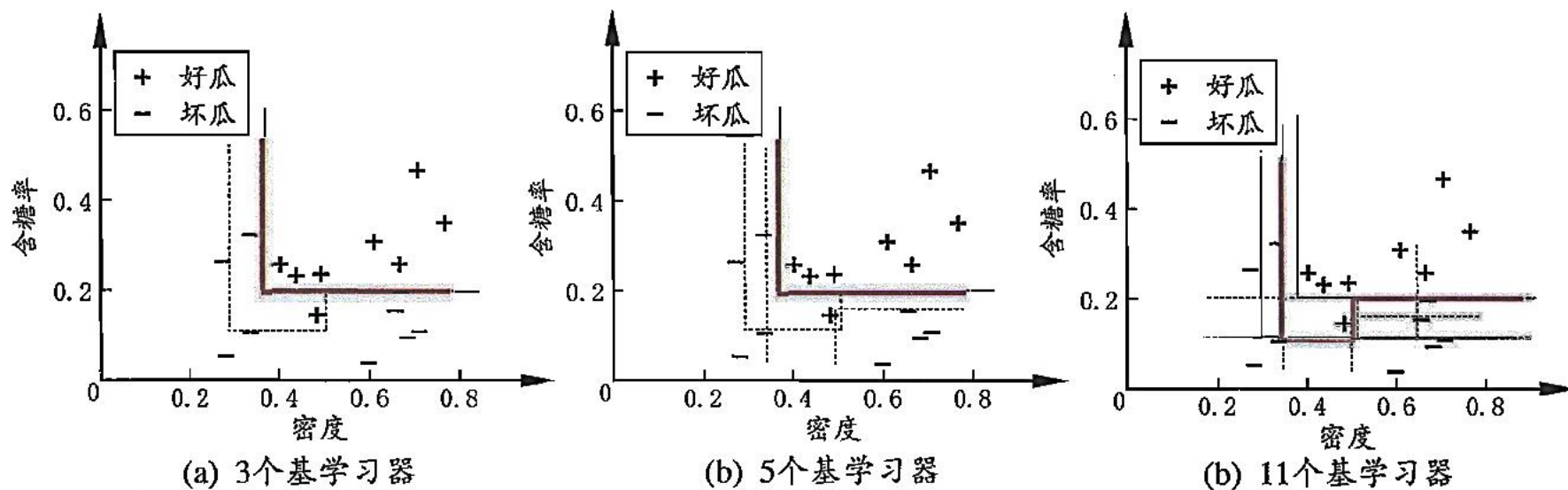


图 8.6 西瓜数据集 3.0 α 上 Bagging 集成规模为 3、5、11 时，集成(红色)与基学习器(黑色)的分类边界。

8.3 Bagging与随机森林

8.3.2 随机森林(RF)

- Bagging的扩展变体，以决策树为基学习器
- 在决策树的训练过程中引入随机属性选择
 - 先从基决策树结点的属性集合中随机选择一个包含 k 个属性的子集，再从中选择最优属性
 - 传统决策树是在当前结点的属性集合(假定有 d 个属性)中选择最优属性
 - 参数 k 控制随机性的引入程度
 - 若令 $k=d$ ，则基决策树的构建与传统决策树相同
 - 若令 $k=1$ ，则是随机选择一个属性用于划分
 - 推荐值 $k=\log_2 d$
- 优势
 - 随机森林对Bagging只做了小改动，简单易实现，计算开销小，性能强大
 - 基学习器多样性来自于样本扰动、属性扰动，提升了个体学习器的差异，增加了集成泛化性能

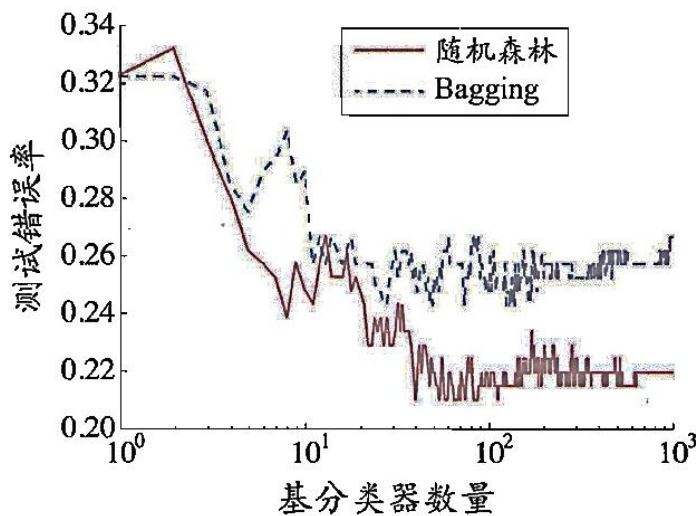
8.3 Bagging与随机森林

8.3.2 随机森林(RF)

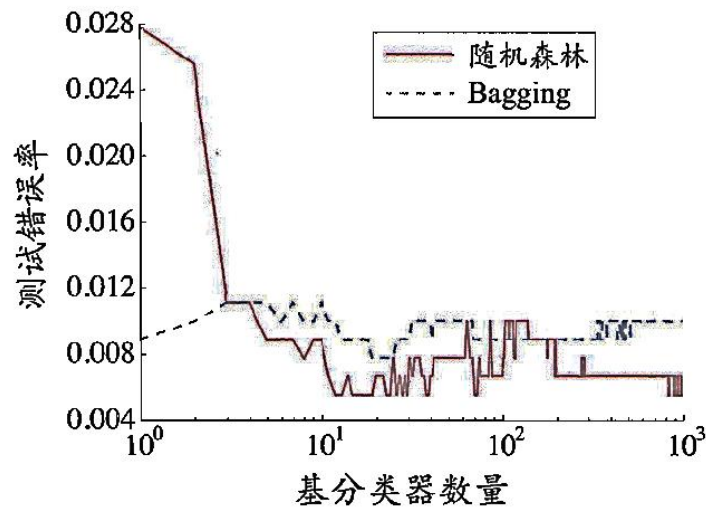
■ 优势

■ 收敛性

- 起始性能相对较差
 - 因为引入属性扰动，个体学习器的性能有所降低
- 通常会收敛到更低的泛化误差
- 训练效率优于Bagging
 - Bagging在选择划分属性时要考虑结点的所有属性
 - 随机森林只考察一个属性子集



(a) glass 数据集



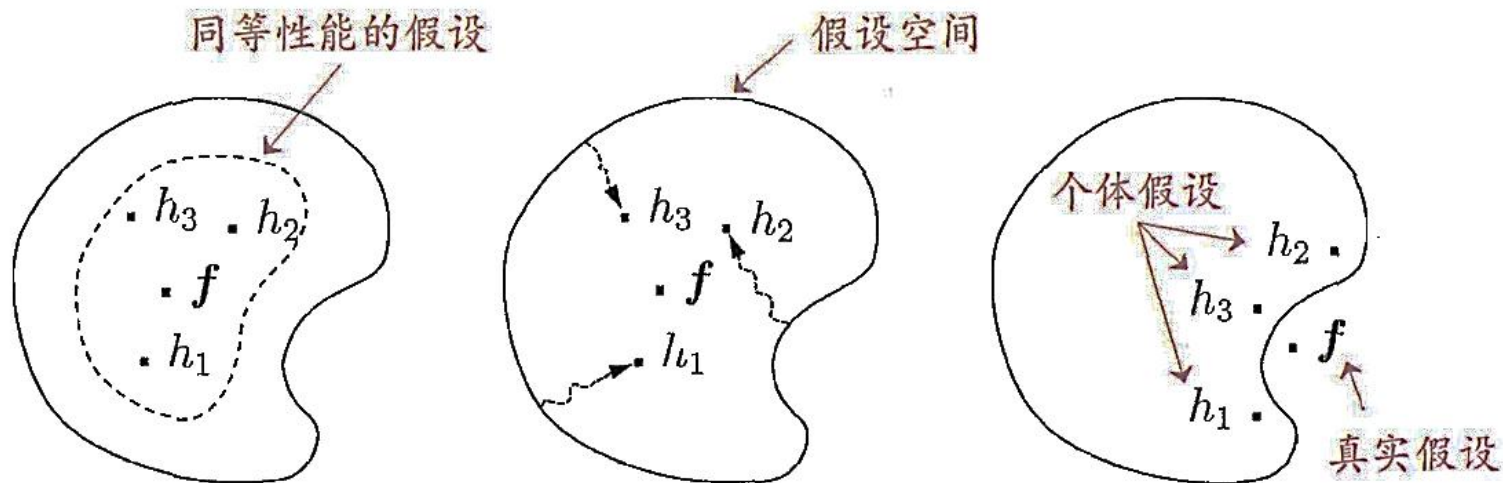
(b) auto-mpg 数据集

8.4 学习器结合策略

结合的优势

■ 三个方面

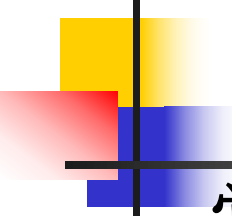
- 从统计看，学习任务假设空间很大，单学习器可能误选而泛化性能不佳，多学习器能减小这风险
- 从计算看，学习算法会陷入局部极小，而通过多次运行后进行结合，可降低陷入局部极小风险
- 从表示看，学习任务的真实假设可能不在当前假设空间，结合多个学习器，相应假设空间会扩大



(a) 统计的原因

(b) 计算的原因

(c) 表示的原因



8.4 学习器结合策略

■ 常用结合方法

- 平均法
 - 简单平均法
 - 加权平均法
- 投票法
 - 绝对多数投票法
 - 相对多数投票法
 - 加权投票法
- 学习法

8.4 学习器结合策略

8.4.1 平均法

- 简单平均法

$$H(\mathbf{x}) = \frac{1}{T} \sum_{i=1}^T h_i(\mathbf{x})$$

- 加权平均法

$$H(\mathbf{x}) = \sum_{i=1}^T \omega_i h_i(\mathbf{x})$$

- 其中 $\omega_i \geq 0$, $\sum_{i=1}^T \omega_i = 1$
- 权重是从训练数据中学习的
 - 现实中的训练样本通常不充分或存在噪声, 使得学出的权重不完全可靠
 - 尤其是对规模比较大的集成, 学习的权重较多, 容易导致过拟合
- 实验应用表明加权平均法不一定优于简单平均法
 - 个体学习器性能相差较大时宜使用加权平均法, 个体学习器性能相近时宜使用简单平均法

8.4 学习器结合策略

8.4.2 投票法

■ 绝对多数投票法

- 某标记得票过半数，则预测为该标记，否则拒绝

$$H(\mathbf{x}) = \begin{cases} c_j, & \text{if } \sum_{i=1}^T h_i^j(\mathbf{x}) > 0.5 \sum_{k=1}^N \sum_{i=1}^T h_i^k(\mathbf{x}) \\ \text{reject}, & \text{otherwise} \end{cases}$$

其中 c_j 是类别标记， h_i^j 是 h_i 在 c_j 上的输出

■ 相对多数投票法

- 预测为得票最多的标记，若多个标记获最高票，则从中随机选取

$$H(\mathbf{x}) = c_{\underset{j}{\operatorname{argmax}} \sum_{i=1}^T h_i^j(\mathbf{x})}$$

■ 加权投票法

$$H(\mathbf{x}) = c_{\underset{j}{\operatorname{argmax}} \sum_{i=1}^T \omega_i h_i^j(\mathbf{x})}$$

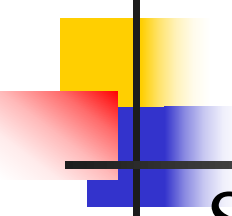
- 其中 $\omega_i \geq 0$, $\sum_{i=1}^T \omega_i = 1$

8.4 学习器结合策略

8.4.2 投票法

■ 特点

- 绝对多数投票法有拒绝预测选项，若学习任务必须提供预测结果，则其将退化为相对多数投票法
- 不同类型个体学习器会产生不同类型的 $h_i^j(\mathbf{x})$
 - 类标记
 - $h_i^j(\mathbf{x}) = \begin{cases} 1, & h_i \text{ 将样本 } \mathbf{x} \text{ 预测为类别 } c_j \\ 0, & \text{否则} \end{cases}$
 - 使用类标记的投票亦称硬投票
 - 类概率
 - $h_i^j(\mathbf{x}) \in [0, 1]$ ，相当于对后验概率 $P(c_j | \mathbf{x})$ 的估计
 - 概率的投票亦称软投票

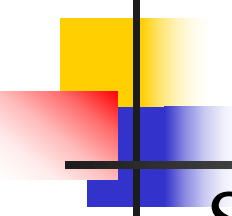


8.4 学习器结合策略

8.4.2 投票法

■ 特点

- 不同类型的 $h_i^j(x)$ 值不能混用
 - 在预测出类别标记的同时产生分类置信度的学习器，其分类置信度可进行规范化转化为类概率
 - 基于类概率结合比直接基于类标记结合性能更好
 - 类概率值不能直接进行比较，可将类概率输出转化为类标记输出再投票



8.4 学习器结合策略

8.4.3 学习法

- 通过另一个学习器结合

- 个体学习器称为初级学习器，用于结合的学习器称为次级学习器或元学习器
- 如Stacking算法

- Stacking方法

- 先从初始数据集训练出初级学习器
- 然后生成一个新数据集用于训练次级学习器
 - 初级学习器的输出作为样例输入特征，初始样本的标记仍作为样例标记

8.4 学习器结合策略

8.4.3 学习法

■ Stacking方法

■ 算法

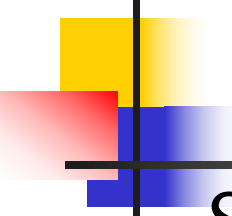
输入：训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
初级学习算法 $\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_T$;
次级学习算法 $\mathcal{L}$.

过程：

```
1: for  $t = 1, 2, \dots, T$  do           使用初级学习算法  $\mathcal{L}_t$   
2:    $h_t = \mathcal{L}_t(D)$ ;               产生初级学习器  $h_t$ .  
3: end for  
4:  $D' = \emptyset$ ;  
5: for  $i = 1, 2, \dots, m$  do  
6:   for  $t = 1, 2, \dots, T$  do  
7:      $z_{it} = h_t(x_i)$ ;  
8:   end for  
9:    $D' = D' \cup ((z_{i1}, z_{i2}, \dots, z_{iT}), y_i)$ ;  
10: end for  
11:  $h' = \mathcal{L}(D')$ ;
```

生成次级训练集.

输出： $H(x) = h'(h_1(x), h_2(x), \dots, h_T(x))$



8.4 学习器结合策略

8.4.3 学习法

■ Stacking方法

■ 特点

- 次级训练集是利用初级学习器产生的，用初级学习器的训练集产生次级训练集，可能过拟合
 - 使用交叉验证或留一法，用训练初级学习器未使用的样本产生次级学习器的训练样本
- 次级学习器的输入属性表示和次级学习算法对Stacking集成的泛化性能有很大影响
 - 有研究表明，将初级学习器的输出类概率作为次级学习器的输入属性，用多响应线性回归(Multi-response Linear Regression)作为次级学习算法效果较好

8.5 多样性

8.5.1 误差-分歧分解

- 欲构建泛化能力强的集成，个体学习器的多样性是集成学习的关键
- 理论分析(回归学习)

- 对示例 $\mathbf{x}$ ，定义学习器 h_i 的分歧为

$$A(h_i|\mathbf{x})=[h_i(\mathbf{x})-H(\mathbf{x})]^2$$

- 集成的分歧

$$\bar{A}(h|\mathbf{x})=\sum_i^T w_i A(h_i|\mathbf{x})=\sum_i^T w_i [h_i(\mathbf{x})-H(\mathbf{x})]^2$$

- 表征个体学习器在样本上的不一致性，即反映个体学习器的多样性

- 个体学习器 h_i 和集成 H 的平方误差分别为

$$E(h_i|\mathbf{x})=[f(\mathbf{x})-h_i(\mathbf{x})]^2$$

$$E(H|\mathbf{x})=[f(\mathbf{x})-H(\mathbf{x})]^2$$

8.5 多样性

8.5.1 误差-分歧分解

■ 理论分析(回归学习)

- 个体学习器误差的加权均值

$$\overline{E}(h|\mathbf{x}) = \sum_{i=1}^T w_i E(h_i|\mathbf{x})$$

- 有

$$\overline{A}(h|\mathbf{x}) = \overline{E}(h|\mathbf{x}) - E(H|\mathbf{x})$$

- 对于全样本

- 个体学习器的泛化误差和分歧为

$$E_i = \int E(h_i|\mathbf{x}) p(\mathbf{x}) d\mathbf{x}, \quad A_i = \int A(h_i|\mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

- 集成的泛化误差

$$E = \int E(H|\mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

- 个体学习器的加权分歧值和泛化误差的加权均值

$$\overline{A} = \sum_{i=1}^T w_i A_i, \quad \overline{E} = \sum_{i=1}^T w_i E_i$$

- 可得

$$E = \overline{E} - \overline{A}$$

- 个体学习器准确性越高、多样性越大，集成越好

8.5 多样性

8.5.1 误差-分歧分解

- 优化 $E = \bar{E} - \bar{A}$ 的困境
 - 优化是在整个样本空间
 - $\bar{A}$ 不是一个可直接操作的多样性度量
 - 仅在集成构造好之后才能进行估计
 - 只适用于回归学习，难以推广到分类学习任务

8.5.2 多样性度量

- 是估算个体学习器的多样化程度
 - 典型是考虑个体分类器的两两相似/不相似程度
 - 一般通过两分类器的预测结果列联表定义
 - a, b, c, d 是样本数目
 - $a + b + c + d = m$

	$h_i = +1$	$h_i = -1$
$h_j = +1$	a	c
$h_j = -1$	b	d

8.5 多样性

	$h_i = +1$	$h_i = -1$
$h_j = +1$	a	c
$h_j = -1$	b	d

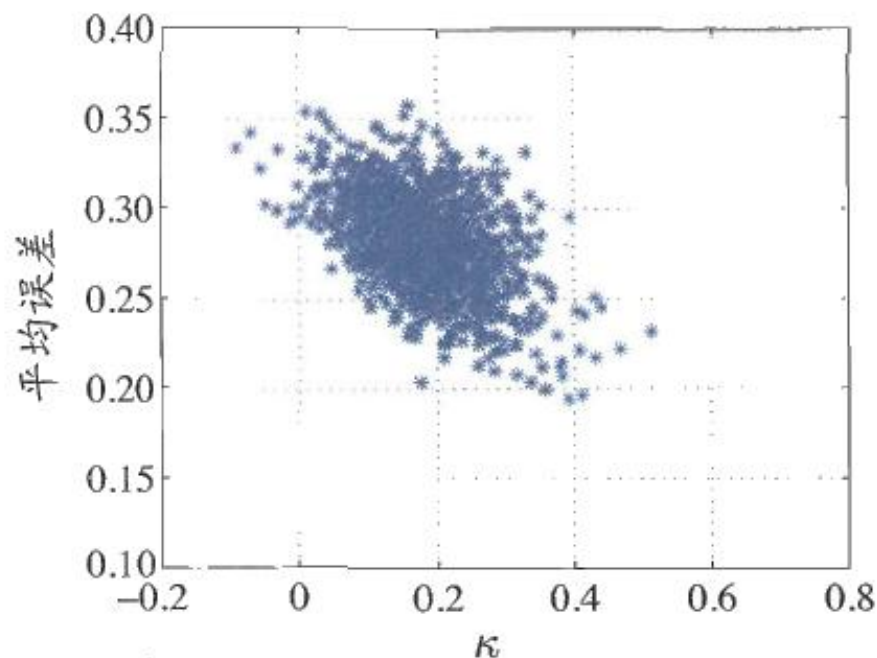
8.5.2 多样性度量

- 不合度量 $dis_{ij} = (b+c)/m$
 - 值域为 $[0, 1]$ ，值越大则多样性越大
- 相关系数 $\rho_{ij} = \frac{ad-bc}{\sqrt{(a+b)(a+c)(c+d)(b+d)}}$
 - 值域 $[-1, 1]$ ，无关为0，正相关为正，否则为负
- Q统计量 $Q_{ij} = (ad-bc)/(ad+bc)$
 - 与相关系数的符号相同，且 $|Q_{ij}| \leq |\rho_{ij}|$
- κ 统计量 $\kappa = (p_1 - p_2)/(1 - p_2)$
 - p_1 是两个分类器取得一致的概率、 p_2 偶然达成一致的概率
$$p_1 = (a+d)/m$$
$$p_2 = [(a+b)(a+c) + (c+d)(b+d)]/m^2$$
 - κ 越大多样性越小

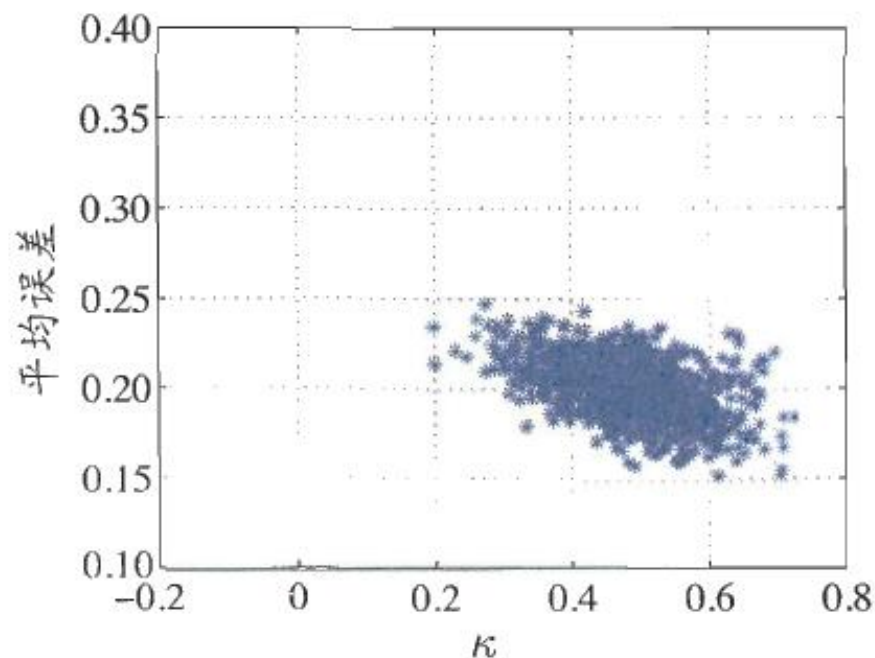
8.5 多样性

8.5.2 多样性度量

- 以上介绍的都是成对型多样性度量，可通过2维图绘制，如

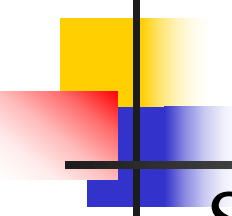


(a) AdaBoost 集成



(b) Bagging 集成

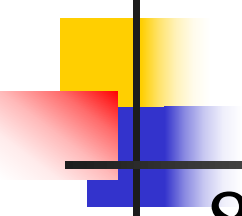
图 8.10 在 UCI 数据集 *tic-tac-toe* 上的 κ -误差图. 每个集成含 50 棵 C4.5 决策树



8.5 多样性

8.5.3 多样性增强

- 一般思路是在学习过程中引入随机性(扰动)
 - 数据样本、输入属性、输出表示、算法参数
- 数据样本扰动
 - 通常基于采样法，简单高效，使用最广
 - 对决策树、神经网络等不稳定基学习器有效。对线性学习器、支持向量机、朴素贝叶斯等稳定基学习器不敏感
- 输入属性扰动
 - 不同属性子空间提供了不同视角，训练出的个体学习器有所不同。如随机子空间算法
 - 对包含大量冗余属性数据，能增强多样性，还能节省时间开销
 - 数据只包含少量属性，或者冗余属性很少，则不宜使用



8.5 多样性

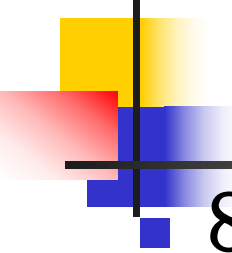
8.5.3 多样性增强

■ 输出表示扰动

- 对输出表示进行操纵以增强多样性
- 翻转法
 - 随机改变一些训练样本的标记，对训练样本的类标记稍作变动
- 输出调制法
 - 对输出表示进行转化，将分类输出转化为回归输出后构建个体学习器
- ECOC法
 - 将原任务拆解为多个子任务，利用纠错输出码将多分类任务拆解为二分类任务训练基学习器

■ 算法参数扰动

- 基学习算法都有参数需设置，如通过随机设置不同参数，可产生差别较大的个体学习器



作业

8.3、8.5